CIFAR-100 CNN Experiment: Local CPU / TensorFlow Results Walkthrough

Deep Learning Course Assignment

This section documents a local CPU-based run of the CIFAR-100 CNN experiment. The model was run as a standalone TensorFlow/Keras script from VS Code / terminal. The purpose was to compare a local CPU execution path with the GPU-based notebook and terminal runs, while keeping the same general CIFAR-100 CNN workflow.

Dataset: CIFAR-100
Framework: TensorFlow / Keras
Execution: Local Python virtual environment through VS Code / terminal
Device: CPU execution, because CUDA drivers were not available for this TensorFlow/Keras run

1. Purpose of this run

This run tested the CIFAR-100 image classification workflow using TensorFlow/Keras on a local CPU environment. The script loaded CIFAR-100, selected a smaller 10-class subset, normalized the images, converted labels to one-hot encoding, trained a convolutional neural network, saved training plots, evaluated the model on the test set, and saved prediction examples.

This report is separate from the Google Colab notebook, the AMD Cloud / ROCm walkthrough, and the local PyTorch CUDA terminal report. It focuses only on the local TensorFlow/Keras CPU run.

2. Execution environment

The script was executed locally through VS Code / terminal. The earlier terminal output showed that TensorFlow could not find CUDA drivers, so GPU acceleration was not used for this run.

```
Could not find cuda drivers on your machine, GPU will not be used.
```

This means the TensorFlow/Keras model trained on CPU. This is useful as a comparison point because CPU training is usually slower than GPU training, especially for convolutional neural networks.

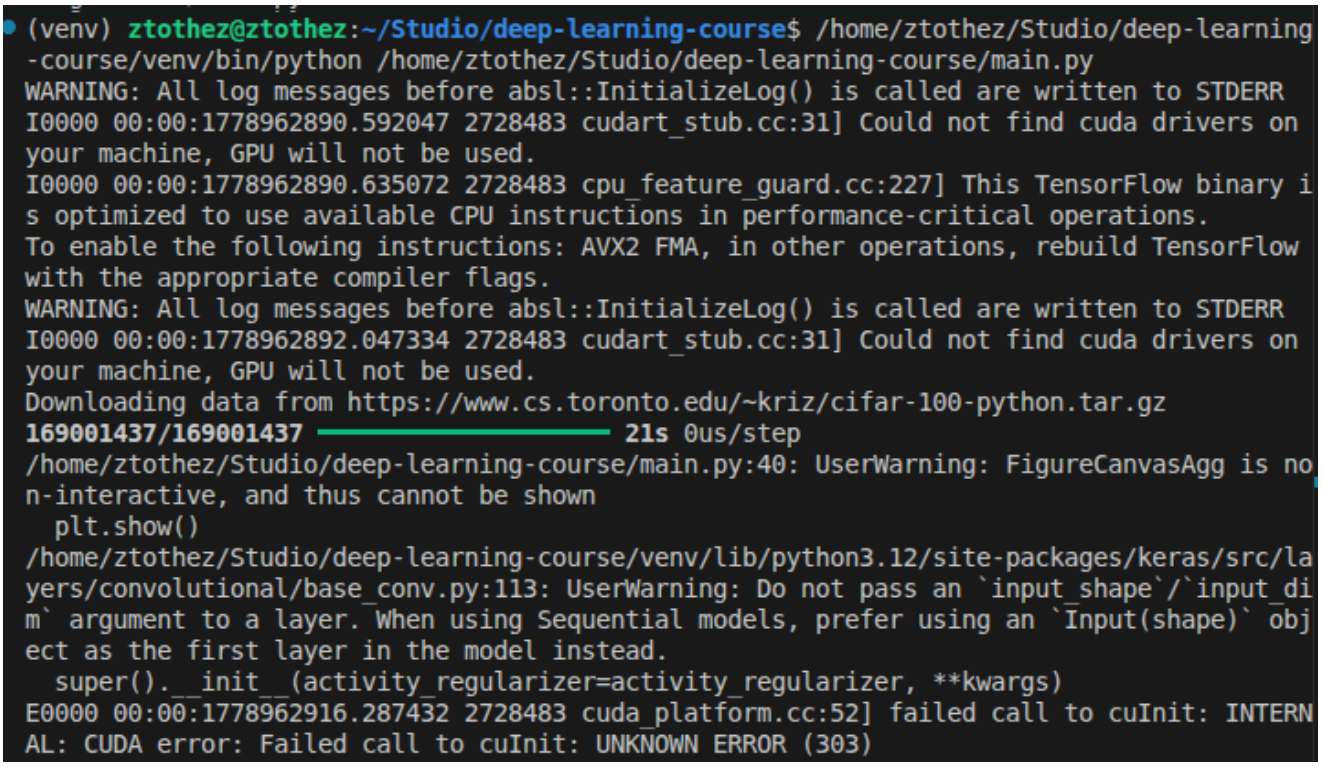

Figure 1. Local TensorFlow/Keras terminal output showing that CUDA drivers were not available, so the CPU was used for this run.

3. Full script used for the CPU run

The full script was run locally as a TensorFlow/Keras Python script. It loads CIFAR-100, filters the dataset to 10 classes, builds a CNN using Keras Sequential layers, trains the model for 30 epochs, saves accuracy/loss curves, evaluates the test set, and saves prediction images.

```
import tensorflow as tf
from tensorflow.keras import datasets, layers, models
from tensorflow.keras.utils import to_categorical
import matplotlib.pyplot as plt
import numpy as np

# Load CIFAR-100 dataset
(train_images, train_labels), (test_images, test_labels) =
datasets.cifar100.load_data()

# Filter to 10 classes
selected_classes = np.arange(10)
class_names = ['apple', 'aquarium_fish', 'baby', 'bear', 'beaver',
               'bed', 'bee', 'beetle', 'bicycle', 'bottle']

train_mask = np.isin(train_labels, selected_classes).flatten()
X_train = train_images[train_mask]
y_train = train_labels[train_mask]

test_mask = np.isin(test_labels, selected_classes).flatten()
X_test = test_images[test_mask]
```

```
y_test = test_labels[test_mask]

# Normalize pixel values to be between 0 and 1
X_train = X_train.astype('float32') / 255.0
X_test = X_test.astype('float32') / 255.0

# Convert labels to one-hot encoding
y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)

# Visualize some training images
plt.figure(figsize=(10, 10))
for i in range(16):
    plt.subplot(4, 4, i+1)
    plt.xticks([])
    plt.yticks([])
    plt.grid(False)
    plt.imshow(X_train[i])
    plt.xlabel(class_names[np.argmax(y_train[i])])
plt.savefig('sample_images.png')

# Build the CNN model
model = models.Sequential()
model.add(layers.Input(shape=(32, 32, 3)))
model.add(layers.Conv2D(32, (3, 3), activation='relu', padding='same'))
model.add(layers.Conv2D(32, (3, 3), activation='relu', padding='same'))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Dropout(0.25))

model.add(layers.Conv2D(64, (3, 3), activation='relu', padding='same'))
model.add(layers.Conv2D(64, (3, 3), activation='relu', padding='same'))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Dropout(0.25))

model.add(layers.Conv2D(128, (3, 3), activation='relu', padding='same'))
model.add(layers.Conv2D(128, (3, 3), activation='relu', padding='same'))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Dropout(0.25))

model.add(layers.Flatten())
model.add(layers.Dense(512, activation='relu'))
model.add(layers.Dropout(0.5))
model.add(layers.Dense(10, activation='softmax'))

model.compile(optimizer='adam',
              loss='categorical_crossentropy',
              metrics=['accuracy'])

model.summary()
```

```
# Train the model
history = model.fit(X_train, y_train,
                    epochs=30,
                    batch_size=64,
                    validation_split=0.2)

# Plot and save results
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.title('Model Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()

plt.subplot(1, 2, 2)
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title('Model Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()

plt.savefig('cpu_baseline_results.png')
print("Plot saved to cpu_baseline_results.png")

# Evaluate on test set
test_loss, test_acc = model.evaluate(X_test, y_test, verbose=2)
print(f"\nFinal test accuracy: {test_acc*100:.2f}%")

# Predict on test images
def plot_predictions(index, filename):
    img = X_test[index]
    true_label = class_names[np.argmax(y_test[index])]
    pred_probs = model.predict(np.expand_dims(img, axis=0))
    pred_label = class_names[np.argmax(pred_probs)]
    plt.imshow(img)
    plt.title(f"True: {true_label} | Pred: {pred_label}")
    plt.axis('off')
    plt.savefig(filename)
    plt.close()

for i in range(5):
    plot_predictions(i, f'cpu_prediction_{i}.png')
```

4. Dataset setup

The script used CIFAR-100 and selected the first 10 classes: `apple`, `aquarium_fish`, `baby`, `bear`, `beaver`, `bed`, `bee`, `beetle`, `bicycle`, and `bottle`.

The selected CIFAR-100 class indices were:

```
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

The image pixel values were normalized to the range 0–1. The labels were converted to one-hot encoding using `to_categorical`, because the Keras model used `categorical_crossentropy` as the loss function.

The script saved a sample image grid as:

```
sample_images.png
```

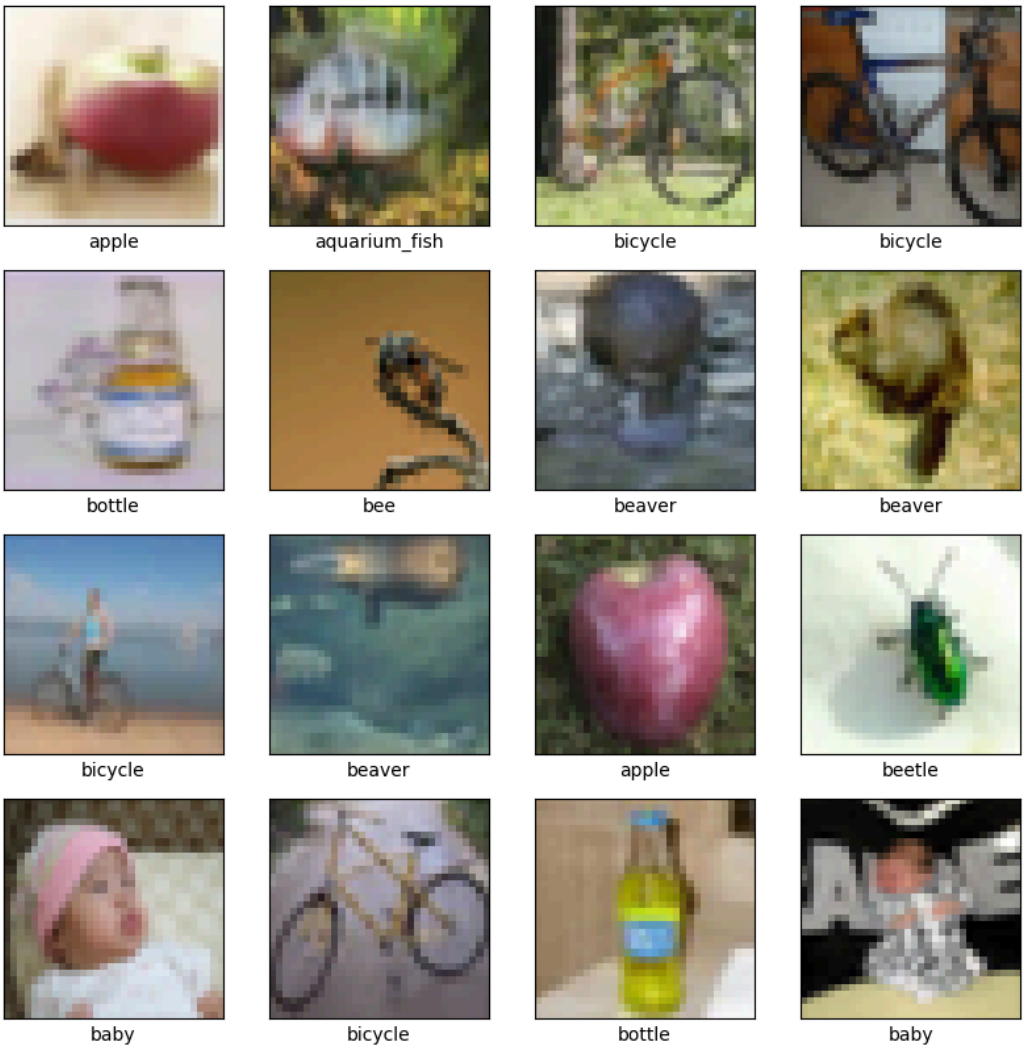

Figure 2. Sample images from the selected 10-class CIFAR-100 subset used in the local

CPU TensorFlow/Keras run.

5. Model and training setup

The CPU model used a Keras Sequential CNN. It had three convolutional blocks with increasing filter sizes: 32, 64, and 128. Each block used two convolutional layers, ReLU activation, max pooling, and dropout. The classifier used a flattened feature vector, a dense layer with 512 units, dropout, and a final softmax output layer for 10 classes.

The main model configuration was:

Component	Setting
Model type	Convolutional Neural Network
Framework	TensorFlow / Keras
Activation	ReLU
Optimizer	Adam
Loss function	Categorical crossentropy
Batch size	64
Epochs	30
Output classes	10
Saved plots	`sample_images.png`, `cpu_baseline_results.png`, `cpu_prediction_*.png`

6. Saved output images from the CPU script

This section contains the image files generated by the TensorFlow/Keras CPU script. These are saved image outputs, not terminal screenshots.

6.1 Training curves

The script saved the training and validation accuracy/loss curves as `cpu_baseline_results.png`.

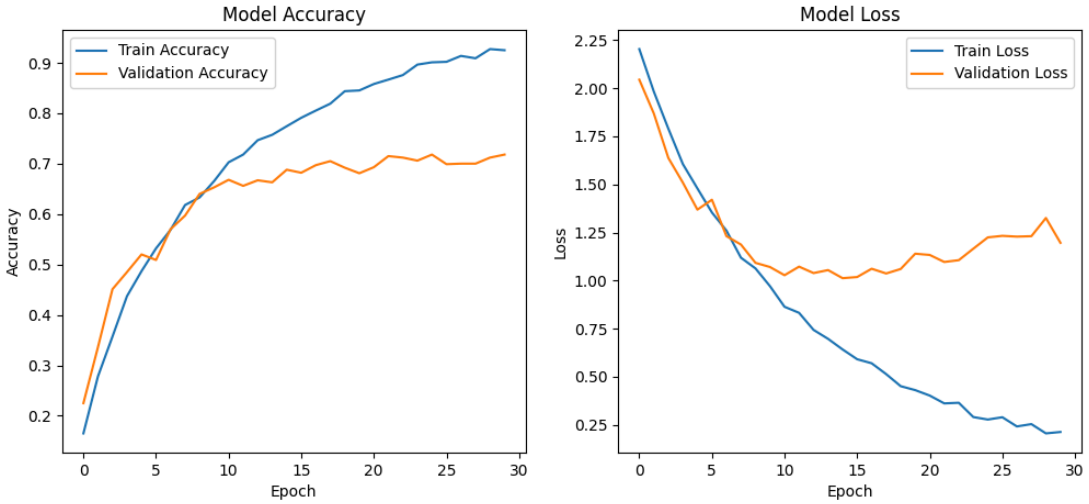

Figure 3. CPU TensorFlow/Keras training and validation accuracy/loss curves saved as `cpu_baseline_results.png`.

The curves show that training accuracy continued to increase, while validation accuracy flattened and validation loss began to rise later in training. This suggests overfitting: the model learned the training data better than it generalized to validation data.

6.2 Prediction examples

The script also saved prediction examples from the test set. These examples show both correct and incorrect classifications.

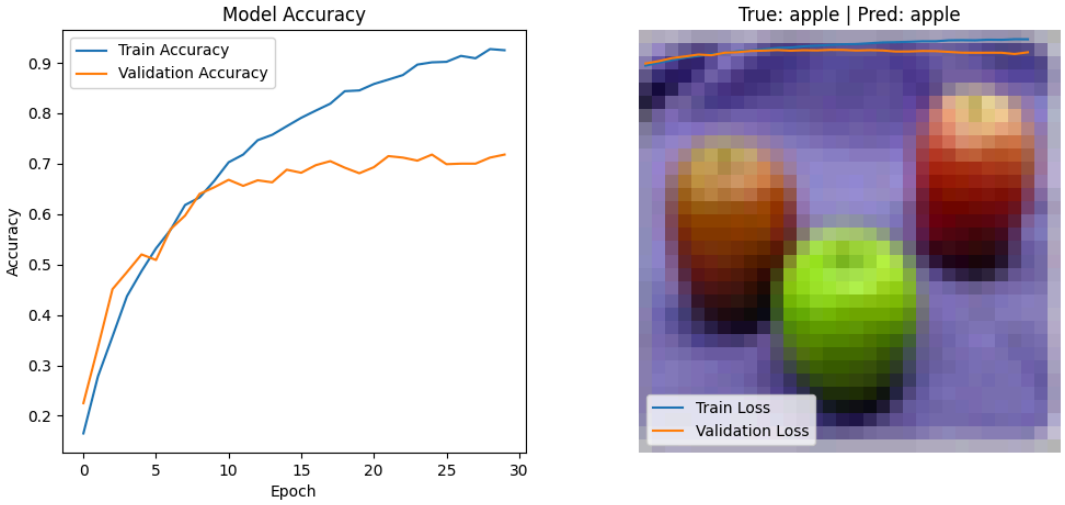

Figure 4a. CPU prediction example where the true class was apple and the model correctly predicted apple.

Figure 4b. CPU prediction example where the true class was bicycle and the model predicted baby.

Figure 4c. CPU prediction example where the true class was beaver and the model

correctly predicted beaver.

Figure 4d. CPU prediction example where the true class was bee and the model correctly predicted bee.

7. Terminal screenshot evidence

This section should contain terminal screenshots only. Use it for evidence that the TensorFlow/Keras script ran locally, used CPU, printed model information, trained for 30 epochs, saved the result plot, and printed the final test accuracy.

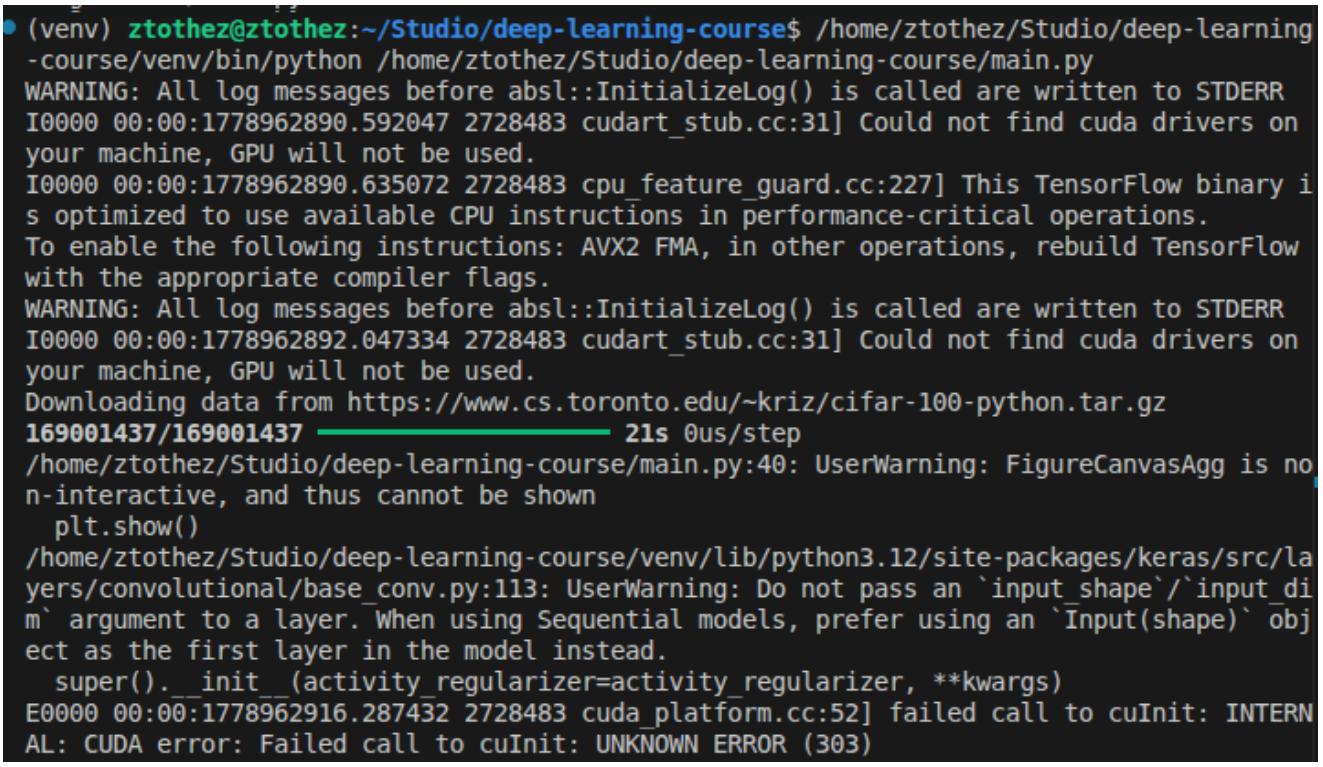

Figure 5. Terminal output showing that CUDA drivers were not found and the TensorFlow/Keras run used CPU.

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 32, 32, 32)	896
conv2d_1 (Conv2D)	(None, 32, 32, 32)	9,248
max_pooling2d (MaxPooling2D)	(None, 16, 16, 32)	0
dropout (Dropout)	(None, 16, 16, 32)	0
conv2d_2 (Conv2D)	(None, 16, 16, 64)	18,496
conv2d_3 (Conv2D)	(None, 16, 16, 64)	36,928
max_pooling2d_1 (MaxPooling2D)	(None, 8, 8, 64)	0
dropout_1 (Dropout)	(None, 8, 8, 64)	0
conv2d_4 (Conv2D)	(None, 8, 8, 128)	73,856
conv2d_5 (Conv2D)	(None, 8, 8, 128)	147,584
max_pooling2d_2 (MaxPooling2D)	(None, 4, 4, 128)	0
dropout_2 (Dropout)	(None, 4, 4, 128)	0
flatten (Flatten)	(None, 2048)	0
dense (Dense)	(None, 512)	1,049,088
dropout_3 (Dropout)	(None, 512)	0
dense_1 (Dense)	(None, 10)	5,130

Total params: 1,341,226 (5.12 MB)
Trainable params: 1,341,226 (5.12 MB)
Non-trainable params: 0 (0.00 B)

Figure 6. Keras model summary showing the CNN layer structure and trainable parameters.

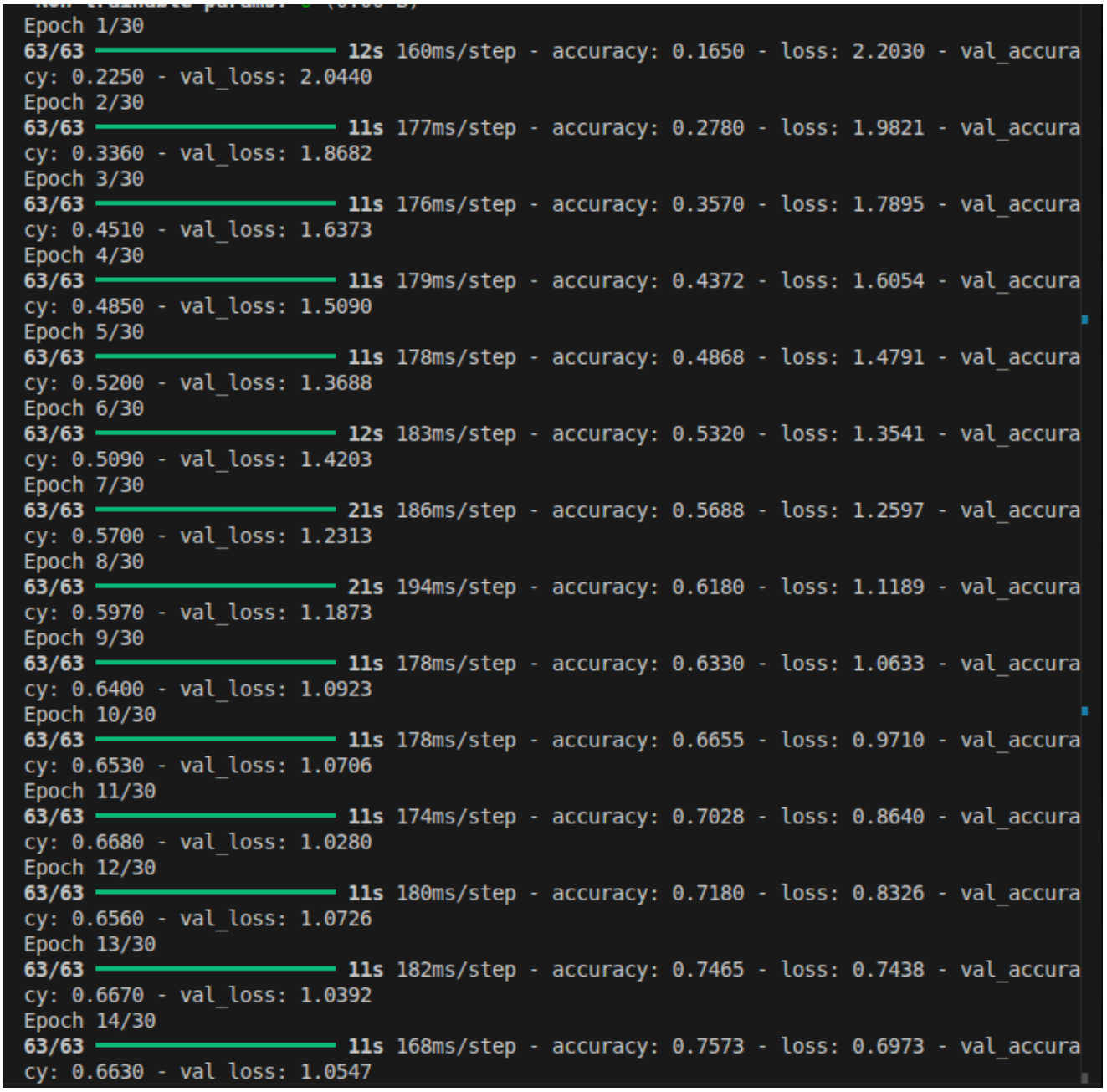

Figure 7. Terminal output showing the CPU model training progress over 30 epochs.

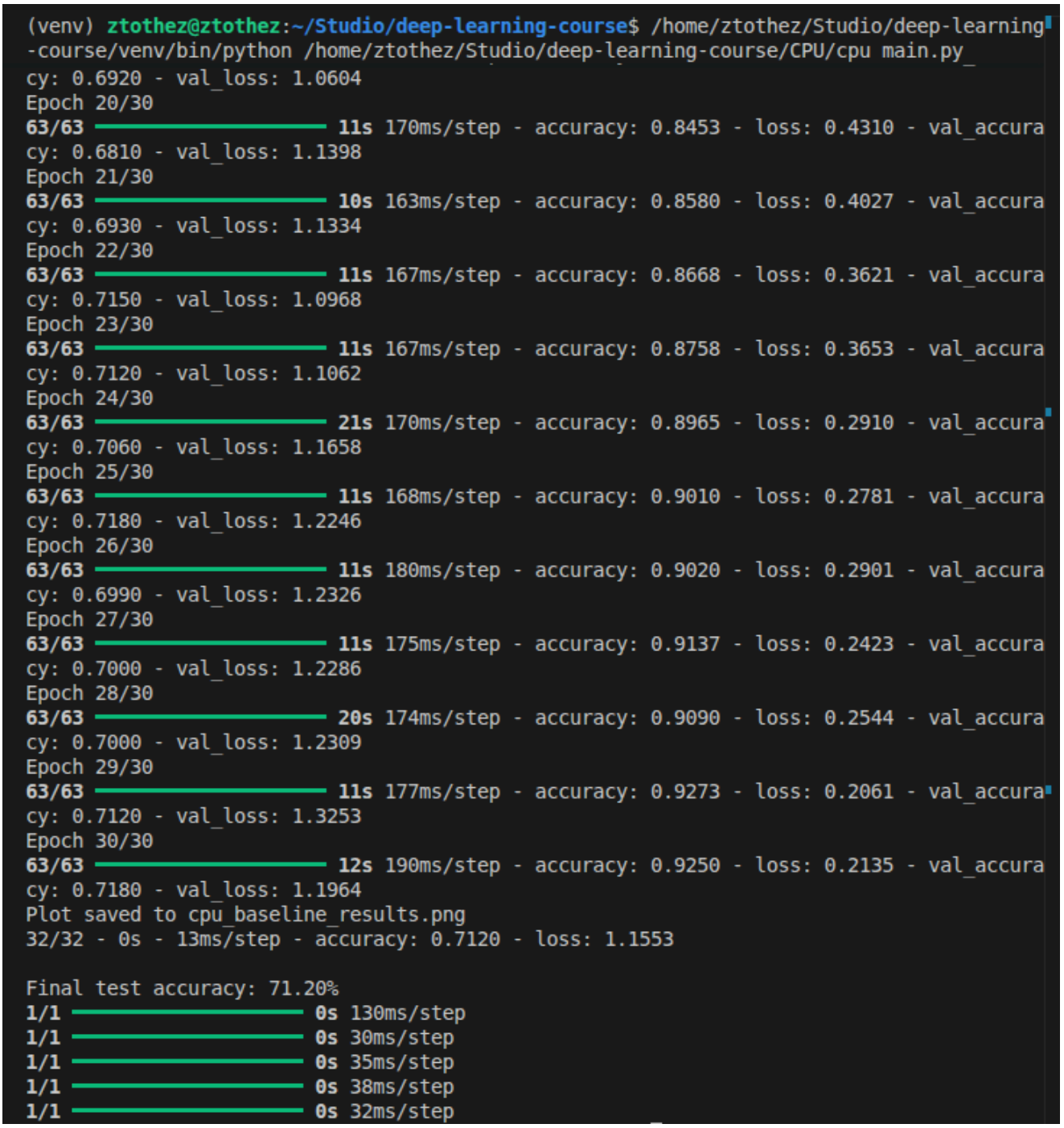

Figure 8. Terminal output showing the final CPU test accuracy after evaluation on the test set.

8. CPU run results

The CPU run trained the CNN for 30 epochs using TensorFlow/Keras. The saved curves show that the model learned from the training data, but validation performance plateaued and validation loss increased later in training.

Result item	Observation
Execution mode	Local TensorFlow/Keras CPU run
Dataset	CIFAR-100, first 10 selected classes
Training setup	ReLU CNN, Adam optimizer, batch size 64

Result item	Observation
Saved result plot	`cpu_baseline_results.png`
Prediction outputs	`cpu_prediction_0.png` to `cpu_prediction_4.png`
Final test accuracy	**[ADD EXACT VALUE FROM TERMINAL OUTPUT]**

```
The final CPU test accuracy was 71.20%.
```

9. Result interpretation

The CPU run confirmed that the CIFAR-100 CNN workflow could be executed locally without GPU acceleration. The model trained successfully, saved output images, and produced prediction examples.

The training curves show overfitting. Training accuracy increased steadily, but validation accuracy flattened later in training. At the same time, validation loss increased, which suggests that the model became more confident on the training data while generalizing less well to validation data.

The prediction examples also show the difficulty of CIFAR-100 classification with small 32 × 32 images. Some predictions were correct, such as apple → apple, beaver → beaver, and bee → bee. Other predictions were incorrect, such as bicycle → baby, showing that the model still confused some visually different or low-resolution examples.

Because this run used CPU, it is mainly useful as a local baseline and workflow validation. GPU-based runs are more suitable for repeated experimentation because they train CNNs much faster.

10. Conclusion

The local CPU TensorFlow/Keras run successfully trained a CNN on a selected 10-class CIFAR-100 subset. The script loaded and filtered the data, normalized the images, trained a convolutional model, saved training plots, evaluated the test set, and generated prediction examples.

The saved plots show that the model learned the training data but began to overfit later in training. The prediction examples show both successful classifications and mistakes. Overall, this CPU run provides a useful local baseline, but GPU-accelerated environments are better suited for faster CNN experimentation and repeated hyperparameter testing.